

Remote Objects

The RMI (Java Remote Method Invocation) system is a mechanism that enables an object on one Java virtual machine to invoke methods on an object in another Java virtual machine. Any object whose methods can be invoked in this way must implement the `java.rmi.Remote` interface. When such an object is invoked, its arguments are marshalled and sent from the local virtual machine to the remote one, where the arguments are unmarshalled and used. When the method terminates, the results are marshalled from the remote machine and sent to the caller's virtual machine.

To make a remote object accessible to other virtual machines, a program typically registers it with the RMI registry.

RMI (Remote Method Invocation)

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM. The RMI provides remote communication between the applications using two objects *stub* and *skeleton*.

Understanding stub and skeleton

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:

stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

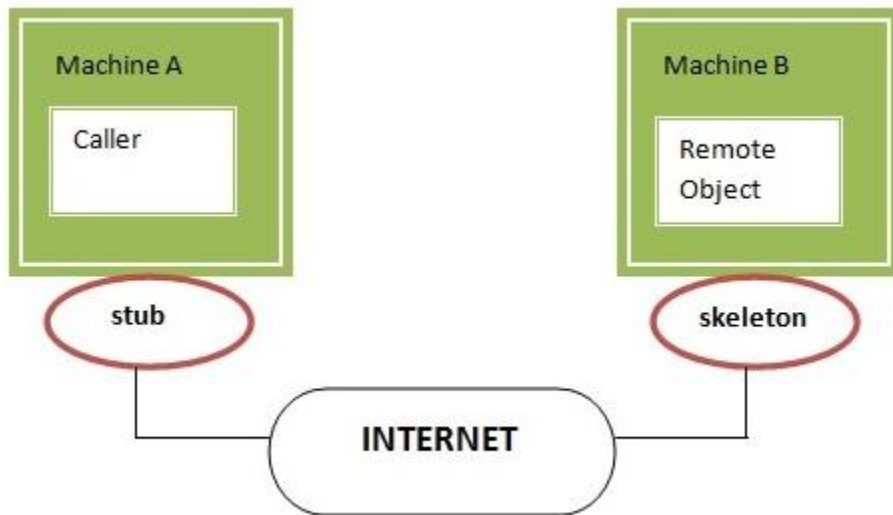
skeleton

The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method

2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, a stub protocol was introduced that eliminates the need for skeletons.



Understanding requirements for the distributed applications

If any application performs these tasks, it can be distributed application.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

The RMI application has all these features, so it is called the distributed application.

Steps to write the RMI program

There are 6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

The RMI Architecture is very simple involving a client program, a server program, a stub and skeleton.

In RMI, the client and server do not communicate directly; instead communicates through **stub** and **skeleton** (a special concept of RMI and this is how designers achieved distributed computing in Java). They are nothing but

special programs generated by **RMI compiler**. They can be treated as **proxies** for the client and server. Stub program resides on client side and skeleton program resides on server. That is, the client sends a method call with appropriate parameters to stub. The stub in turn calls the skeleton on the server. The skeleton passes the stub request to the server to execute the remote method. The return value of the method is sent to the skeleton by the server. The skeleton, as you expect, sends back to the stub. Finally the return value reaches client through stub.

Note: The method existing on the server is a remote method to the client (because client and server may be far away anywhere in the globe); but for the server, it is local method only. So, for the client, the server is remote server (okay, in server point of view, the client is remote client, but in RMI, we do not call client as remote client), a program on the remote server is remote program and finally the method in the remote program is remote method. Know the meaning of remote server, remote program, remote method; this must be very clear to the beginner else the RMI concept is very confusing at the outset. So, anything residing on server is remote to client.

The RMI Architecture process is explained figuratively as under.

As the figure illustrates, there comes three layers in the RMI Architecture – Application layer, Proxy layer and Remote reference layer (Transport layer is part of Remote reference layer).

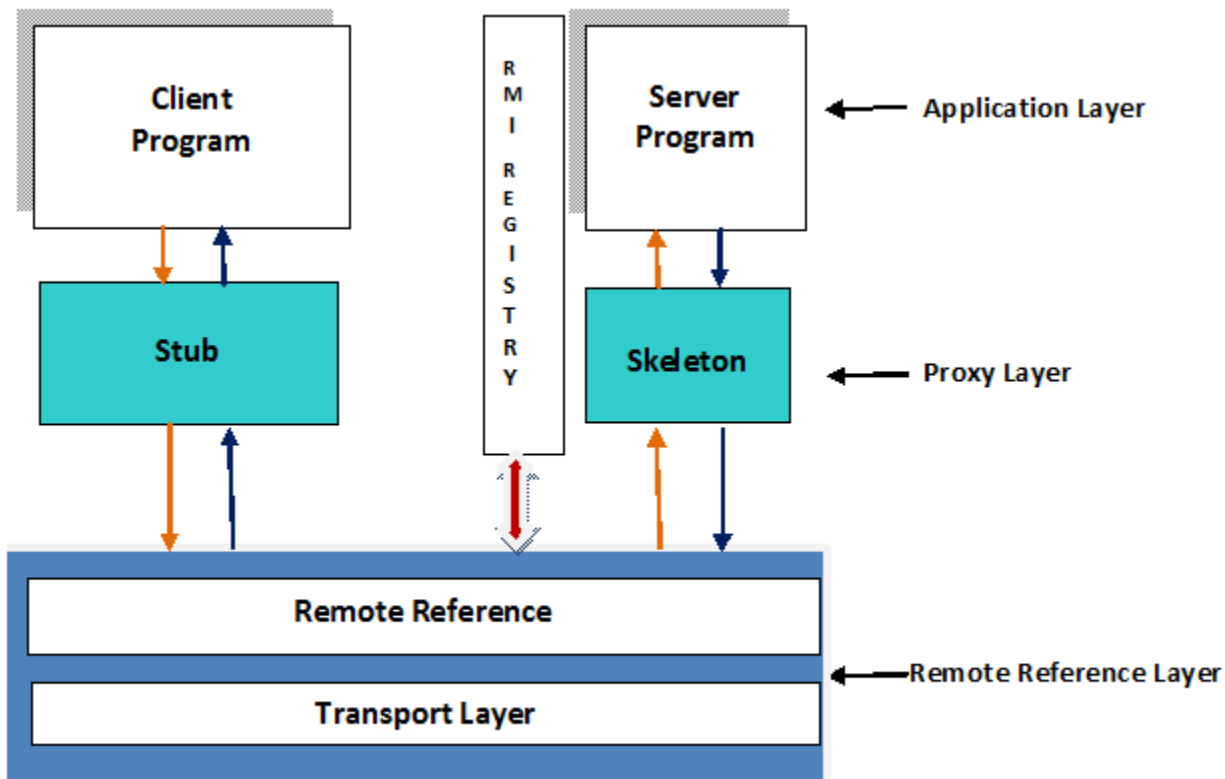


Fig: RMI Architecture

1. Application Layer

This layer is nothing but the actual systems (client and server) involved in communication. A client Java program communicates with the other Java program on the server side. RMI is nothing but a communication between two JVMs placed on different systems.

2. Proxy Layer

The proxy layer consists of **proxies** (named as **stub** and **skeleton** by designers) for client and server. **Stub is client side proxy** and **Skeleton is server side proxy**. Stub and skeleton, as many people confuse, are not separate systems but they are after all programs placed on client and server. The stub and skeleton are not hard coded by the Programmer but they are generated by RMI compiler (this is shown in execution part later). Stub is placed on client side and skeleton is placed on server side. The client server communication goes through these proxies. Client sends its request of method invocation (to be executed on remote server) to stub. Stub inturn sends the request to skeleton. Skeleton passes the request to the server program. Server executes the method and sends the return value to the skeleton (to route to client). Skeleton sends to stub and stub to client program.

Marshaling and Unmarshaling

One more job of proxies is marshaling and unmarshaling. **Marshaling** is nothing but converting data into a special format suitable to pass through the distributed environment without losing **object persistence**. For this reason, the RMI mechanism implicitly serializes the objects involved in communication. The stub marshals the data of client and then sends to the skeleton. As the format is not understood by the server program, it is unmarshaled by the skeleton into the original format and passed to server. Similarly from server to client also.

A marshal stream includes a stream of objects that are used to transport parameters, exceptions, and errors needed for these streams for communicating with each other. Marshaling and unmarshaling are done by the RMI runtime mechanism implicitly without any programmers extra coding.

3. Remote Reference Layer

Proxies are implicitly connected to RMI mechanism through Remote reference layer, the layer responsible for object communication and transfer of objects between client and server. It is responsible for dealing with semantics of remote invocations and implementation – specific tasks with remote objects. In this layer, actual implementation of communication protocols is handled.

4. Transport layer does not exist separately but is a part of Remote reference layer. Transport layer is responsible for actually setting up connections and handling the transport of data from one machine to another. It can be modified to handle encrypted streams, compression algorithms and a number of other security/performance related enhancements.

The marshaled stream is passed to **RRL** (Remote Reference Layer). Task of RRL is to identify the host, that is, remote machine. The marshaled stream is passed to Transport layer which performs routing.

Q. Create a simple RMI Client/Server Java program. The server returns **Hello World FROM RMI SERVER** from a **getMessage()** method. When the client program call the **getMessage()** method it should be printed on the screen in client Program.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface RmiServerIntf extends Remote {
    public String getMessage() throws RemoteException;
}

import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
```

```
import java.rmi.registry.*;
public class RmiServer extends UnicastRemoteObject implements RmiServerIntf {
    public static final String MESSAGE = "Hello World FROM RMI SERVER";

    public RmiServer() throws RemoteException {
        super(0);    // required to avoid the 'rmic' step, see below
    }

    public String getMessage() {
        return MESSAGE;
    }

    public static void main(String args[]) throws Exception {
        System.out.println("RMI server started");

        try { //special exception handler for registry creation
            LocateRegistry.createRegistry(1099);
            System.out.println("java RMI registry created.");
        } catch (RemoteException e) {
            //do nothing, error means registry already exists
            System.out.println("java RMI registry already exists.");
        }

        //Instantiate RmiServer

        RmiServer obj = new RmiServer();

        // Bind this object instance to the name "RmiServer"
        Naming.rebind("//192.168.20.3/RmiServer", obj);
        System.out.println("PeerServer bound in registry");
    }
}

import java.rmi.Naming;

public class RmiClient {
    public static void main(String args[]) throws Exception {
        RmiServerIntf obj = (RmiServerIntf)Naming.lookup("//192.168.20.3/RmiServer");
        System.out.println(obj.getMessage());
    }
}
```

Q. Write a simple RMI Client server program to invoke remote method ***calculate (double, double, double)*** as parameters and which will return Simple interest. Now create a client to call that method and print the interest calculated by server.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface InterestCalculator extends Remote {

    public double calculate(double p, double t, double r) throws RemoteException;
}
```

```
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.registry.*;

public class CalculatorServer extends UnicastRemoteObject implements InterestCalculator {

    public CalculatorServer() throws RemoteException {
        super(0);    // required to avoid the 'rmic' step, see below
    }

    public static void main(String args[]) throws Exception {
        System.out.println("RMI server started");

        try { //special exception handler for registry creation
            LocateRegistry.createRegistry(1099);
            System.out.println("java RMI registry created.");
        } catch (RemoteException e) {
            //do nothing, error means registry already exists
            System.out.println("java RMI registry already exists.");
        }

        //Instantiate RmiServer

        CalculatorServer obj = new CalculatorServer();

        // Bind this object instance to the name "RmiServer"
        Naming.rebind("192.168.20.8/CalculatorServer", obj);
        System.out.println("PeerServer bound in registry");
    }

    public double calculate(double p, double t, double r) throws RemoteException {
        return (p*t*r/100);
    }
}

import java.rmi.Naming;

public class CalculatorClient {
    public static void main(String args[]) throws Exception {
        InterestCalculator obj =
        (InterestCalculator)Naming.lookup("192.168.20.8/CalculatorServer");
        System.out.println("SIMPLE INTEREST: "+obj.calculate(5000, 1, 12));
    }
}
```

Q. Write a RMI Client Server program. The server program is responsible to serve like Music Server when ever user pass the 10 digit CODE to the server method the server will return the equivalent music NAME of your passed CODE. The client passes 10 Digit code to the server and Display the name Music NAME to the client end.

```

import java.rmi.*;
public interface MInterface extends Remote{
    String service(String code)throws RemoteException;
}

import java.rmi.Naming;
import java.rmi.*;
import java.rmi.server.*;
import java.rmi.registry.*;

public class MServer extends UnicastRemoteObject implements MInterface{

    public MServer() throws RemoteException{
        super(0);
    }
    public String service(String str){
        String serv =null;
        if(str.length()!=10){
            serv= "INVALID CODE!";
        }
        switch(str){
            case "1020304051":
                serv="Commedy Song!";
                break;
            case "1020304052":
                serv="Des Vakt Song!";
                break;
            case "1020304053":
                serv="Teej Song!";
                break;
            case "1020304054":
                serv="Lok Dohari Song!";
                break;
            case "1020304055":
                serv="English Song!";
                break;
            default :
                serv ="INVALID CODE!";
        }
        return serv;
    }

    public static void main(String ar[]){
        try{
            LocateRegistry.createRegistry(1099);
            System.out.println("RMI Registry Created!");

            MServer s = new MServer();
            Naming.rebind("192.168.20.185/MS", s);
            System.out.println("Ready...");

        }catch(Exception e){
            System.out.println(e);
        }
    }
}

```

```

    }
}

import java.rmi.Naming;
public class MClient{
    public static void main(String ar[]){
        try{
            MInterface ms =(MInterface)Naming.lookup("192.168.20.185/MS");
            System.out.println(ms.service(ar[0]));
        }catch(Exception e){

        }
    }
}

```

Q) Write a RMI Client/Server Program to calculate Profit or Loss through the help of Selling Price and Cost Price send by Client.

```

import java.rmi.*;
public interface PLInterface extends Remote {
    public double CalculatePL (double c, double s)throws RemoteException;
}

import java.rmi.*;
import java.rmi.registry.*;
import java.rmi.server.UnicastRemoteObject;

public class PLServer extends UnicastRemoteObject implements PLInterface {
    public PLServer() throws RemoteException{
        super(0);
    }

    @Override
    public double CalculatePL(double cp,double sp){
        return (sp-cp);
    }

    public static void main(String args[]){
        try{
            LocateRegistry.createRegistry(1099);
            System.out.println("Registry Created");

            PLServer s = new PLServer();

            Naming.rebind("localhost/PLServer",s);
            System.out.println("Bind Successfully");
        }catch(Exception e){
            System.out.println("Already created");
        }
    }
}

```

```

import java.rmi.Naming;

```

```
public class PLClient {
    public static void main(String args[]){
        try{
            PLInterface PL = (PLInterface)Naming.lookup("localhost/PLServer");

            double P1 = PL.CalculatePL(3200000.33,90000.23);

            if(P1>0){
                System.out.println("Profit");
            }
            else{
                System.out.println("Loss");
            }
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```

Q. Write a program to greeting to Nepal to the calling parameters

```
import java.rmi.RemoteException;
public interface Nepal {
    String greeting(String message) throws RemoteException;
}

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class WelcomeNepal extends UnicastRemoteObject implements Nepal{
    public WelcomeNepal()throws RemoteException{
        super(0);
    }
    @Override
    public String greeting(String message)throws
    RemoteException{
        return ("Welcome to Nepal: "+message);
    }
}

import java.rmi.Naming;
import java.rmi.registry.*;
public class RMIServer{
    public static void main(String args[]){
        try{
            WelcomeNepal nepal = new WelcomeNepal();
            LocateRegistry.createRegistry(1099);
            System.out.println("Registry Created...");

            Naming.rebind("127.0.0.1/greeting",nepal);
            System.out.println("RMI Server Ready...");

        }catch(Exception e){
            System.out.println("SERVER ERROR: "+e);
        }
    }
}
```

```

}

import java.rmi.Naming;
public class RMIClient{
    public static void main(String args[]){
        try{
            Nepal nepal= (Nepal)
                Naming.Lookup("127.0.0.1/greeting");
            System.out.println("RMI Server Reply: "+
                nepal.greeting("SUNIL"));
        }catch(Exception e){
            System.out.println("SERVER ERROR: "+e);
        }
    }
}

```

Q. Write a RMI client server program to say hi to the calling parameter

```

import java.rmi.Remote;
import java.rmi.RemoteException;
public interface RMIDemo extends Remote {
    public String doCommunicate(String name) throws RemoteException;
}

```

```

import java.rmi.RemoteException;
public class RMIDemoImpl extends UnicastRemoteObject implements RMIDemo {
    private static final long serialVersionUID = 1L;
    protected RMIDemoImpl() throws RemoteException {
        super();
    }
    public String doCommunicate(String name) throws RemoteException {
        return "\nServer says: Hi "+name+ "\n";
    }
}

```

```

import java.rmi.Naming;
public class RMIDemoServer {
    public static void main(String[] args) throws Exception {
        RMIDemoImpl rMIDemoImpl = new RMIDemoImpl();
        Naming.rebind("RMIDemo", rMIDemoImpl);
        System.out.println("RMIDemo object bound to the name 'RMIDemo' and is ready for use...");
    }
}

```

```
import java.rmi.Naming;
public class RMIDemoClient {
    public static void main(String[] args) throws Exception {
        if (args.length == 2) {
            String url=new String("rmi://" +args[0]+"/RMIDemo");
            RMIDemo rMIDemo = (RMIDemo)Naming.lookup(url);
            String serverReply = rMIDemo.doCommunicate(args[1]);
            System.out.println("Server Reply: " +serverReply);
        } else {
            System.err.println("Usage: RMIDemoClient <server> <name>");
        }
    }
}
```